

TAREA 3 · PROGRAMACIÓN II

Integración del Proyecto con Base de Datos

Sistema de Ventas en Java Swing (MVC) · Persistencia con JDBC

Josue Zetino

Curso: Programación 2

Fecha: 07 Agosto 2026

Repositorio <https://github.com/jzetinob/sistema-ventas>

Objetivo de la tarea

Reemplazar el almacenamiento en memoria y archivos CSV por una base de datos relacional usando JDBC, de modo que los registros se guarden de forma permanente.

Requisitos mínimos

1

Configurar la conexión

Cargar el driver JDBC y abrir la conexión desde la aplicación.

2

Crear las tablas

Generar el esquema necesario para clientes, productos y facturas.

3

Implementar la persistencia

Sustituir el acceso a CSV por operaciones SQL desde las clases DAO.

4

Guardar desde los formularios

Que lo capturado en las vistas Swing termine en la base de datos.

5

Verificar permanencia

Cerrar y reabrir la aplicación comprobando que los datos siguen ahí.

Base de datos seleccionada: SQLite

Motor relacional ligero que no requiere servidor: la base es un único archivo que la aplicación crea sola.

Sin servidor

No hay que instalar ni administrar un motor aparte; todo corre dentro de la aplicación.

Un solo archivo

La base vive en `datos/sistema_ventas.db` y se crea automáticamente al arrancar.

SQL estándar

Soporta llaves primarias, llaves foráneas, restricciones UNIQUE y transacciones.

Controlador JDBC agregado como dependencia Maven

```
<dependency>
  <groupId>org.xerial</groupId>
  <artifactId>sqlite-jdbc</artifactId>
  <version>3.47.1.0</version>
</dependency>
```

Tablas creadas

Al arrancar, la aplicación ejecuta CREATE TABLE IF NOT EXISTS y genera las cuatro tablas del sistema.

clientes

- **id** — PK autoincremental
- nit — TEXT único
- nombre
- direccion
- telefono

productos

- **id** — PK autoincremental
- codigo — TEXT único
- nombre
- precio

facturas

- **id** — PK autoincremental
- numero_factura — único
- nit · cliente
- fecha
- total

factura_detalle

- **id** — PK
- factura_id — FK → facturas
- ON DELETE CASCADE
- producto · cantidad
- precio · subtotal

Integridad referencial: factura_detalle apunta a facturas mediante una llave foránea con ON DELETE CASCADE, de modo que al borrar una factura desaparecen también sus líneas de detalle.

Cómo se realizó la conexión

La clase ConexionBD aplica el patrón singleton: carga el driver y abre la conexión una sola vez.

```
Class.forName("org.sqlite.JDBC");  
  
Connection conexion = DriverManager  
    .getConnection(  
        "jdbc:sqlite:datos/sistema_ventas.db");
```

Singleton

Una única instancia de conexión para toda la aplicación.

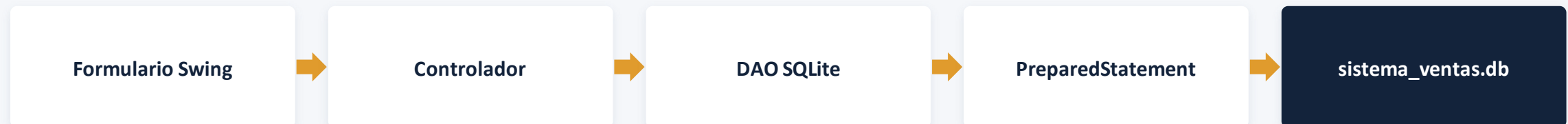
Se abre al iniciar

Se establece una vez y se reutiliza en cada operación.

Ruta relativa

El archivo .db se guarda en la carpeta datos/.

Flujo de una operación



Clases agregadas para la persistencia

ConexionBD

Administra la conexión y crea las tablas al arrancar la aplicación.

ClienteDAOSQLite
ProductoDAOSQLite
FacturaDAOSQLite

Implementan el CRUD con PreparedStatement, lo que protege contra inyección SQL, y recuperan el id generado con RETURN_GENERATED_KEYS.

MigradorDatos

Traslada a la base de datos los registros que existían en los CSV de versiones anteriores. Solo se ejecuta la primera vez.

Un cambio de una sola línea por controlador

Las nuevas clases implementan las mismas interfaces DAO que usaba la versión con CSV, así que las vistas y los controladores no se tocaron.

```
dao = FacturaDAOSQLite.getInstance();  
// antes: dao = FacturaDAOCsv.getInstance();
```

Detalles de implementación

1

Transacciones

La factura y sus detalles se guardan dentro de una transacción con `setAutoCommit(false)` más `commit` o `rollback`: o se guarda completa, o no se guarda nada.

```
setAutoCommit(false) → commit / rollback
```

2

Correlativo desde la BD

El siguiente número de factura (FAC-0001, FAC-0002...) se calcula con `MAX` sobre la tabla facturas, sin depender de un contador externo en archivo.

```
SELECT MAX(numero_factura) FROM facturas
```

3

Borrado en cascada

Al eliminar una factura, la llave foránea con `CASCADE` borra automáticamente todas sus líneas de detalle asociadas.

```
FOREIGN KEY ... ON DELETE CASCADE
```

Sistema de Ventas

ArchivoCatálogosEdiciónVentanaAyuda

Formulario de Facturación

Datos del Cliente

Cliente:

Fecha:

2026-08-07

NIT:

No. Factura:

FAC-0004

Cliente del catálogo:

93754578345 - James

Agregar Producto

Producto:

Cantidad:

Precio:

Agregar

Producto	Cantidad	Precio	Subtotal

Catálogo de Productos

Datos del Producto

Código:

Guardar

Nombre:

Actualizar

Precio:

Eliminar

Limpiar

No.	Código	Nombre	Precio
1	900	laptop	5350.00
2	901	Mouse	55.00
3	902	Teclado	95.00
4	903	cargador 12v	150.00
6	904	mini pc	2500.00

Facturas Registradas

No.	NIT	Cliente	Fecha	Total
1	876577509	Vanesa	2026-08-07	435.00
2	876577509	Vanesa	2026-08-07	550.00
4	93754578345	James	2026-08-07	17500.00

Ver Detalle

Eliminar

Actualizar

Cerrar

Catálogo de Clientes

Datos del Cliente

NIT:

Guardar

Nombre:

Actualizar

Dirección:

Eliminar

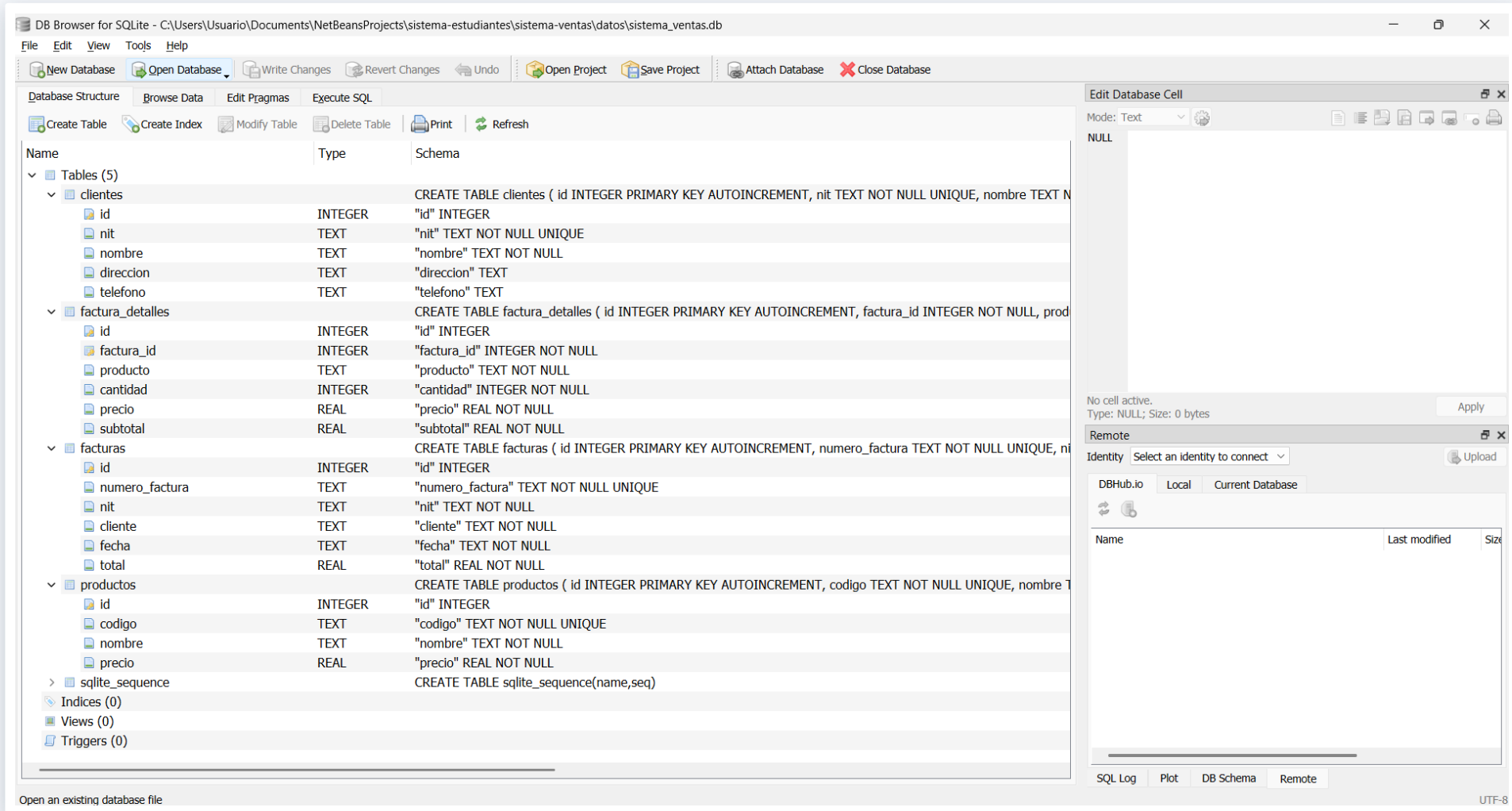
Teléfono:

Limpiar

No.	NIT	Nombre	Dirección	Teléfono
1	975487363	Alberto	Ciudad Guatemala	9697923654
2	93754578345	James	Escuintla, Palín	87656578
3	876577509	Vanesa	Amatitlan, Guatemala	7898009843

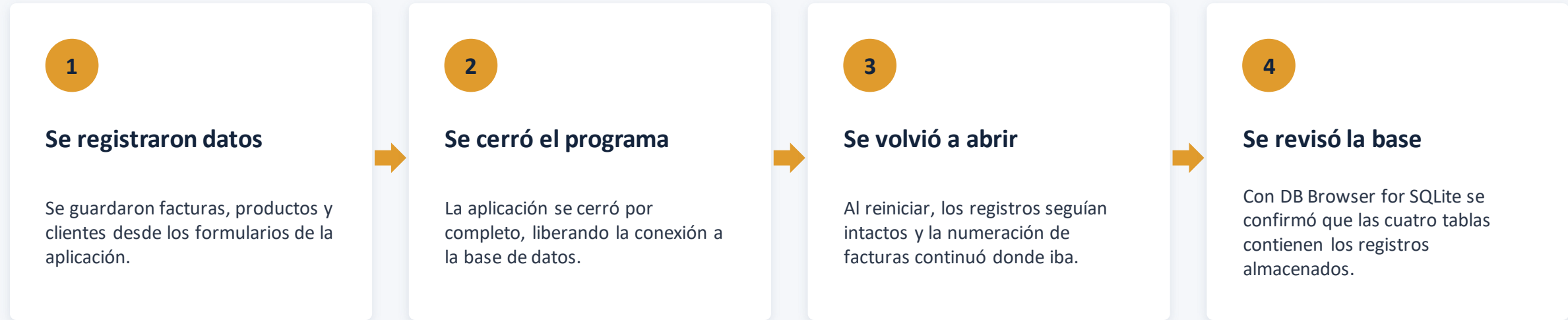
Base de datos con registros

DB Browser for SQLite mostrando el esquema de las cuatro tablas dentro de sistema_ventas.db.



Verificación de persistencia

Prueba realizada para comprobar que los datos sobreviven al cierre de la aplicación.



Resultado: la información dejó de perderse al cerrar el programa. La base de datos conserva clientes, productos, facturas y sus detalles entre sesiones.

Conclusiones

1

La arquitectura DAO probó su valor

Cambiar toda la persistencia no obligó a modificar las vistas ni los controladores: bastó una línea por controlador.

2

SQLite garantiza permanencia e integridad

Unicidad de NIT, código y número de factura, más llaves foráneas que mantienen la coherencia entre facturas y detalles.

3

Las transacciones evitan datos a medias

Una factura nunca queda guardada sin sus líneas de detalle: o entra completa, o se revierte.

Repositorio público

<https://github.com/jzetinob/sistema-ventas>